Functional And Technical Design Document

Team 1D

Year 1 Period 3

Table of contents

1. Introduction

- 1.1 Purpose of the Document
- 1.2 Scope of the System
- 1.3 Client / Stakeholders

2. Project Overview

- 2.1 System Description
- 2.2 Goals & Objectives
- 2.3 Development and Testing Platform

3. Requirements

- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 Constraints & Assumptions
- 3.4 Requirement Prioritization

4. System Architecture

- 4.1 Architecture Overview
- 4.2 High-Level Design

5. Hardware Design

- 5.1 Robot Components
- 5.2 Power System
- 5.3 Hardware Constraints
- 5.4 I/O List

6. Software Design

- 6.1 Software Architecture
- 6.2 Modules & Components
- 6.3 Control Logic

7. Processes & Workflows

- 7.1 Battle Scenario Flow

8. Testing & Validation

- 8.1 Test Strategy

9. Standards & Guidelines

- 9.1 Coding Standards
- 9.2 Safety Requirements

10. References to Related Documents

1. Introduction

1.1 Purpose of the Document

The purpose of this document is to provide a detailed technical design of the *Battle Bot* system. It describes the architecture, hardware and software components, and the overall structure required to develop, integrate, and operate the robot.

This document serves as a reference for developers and project stakeholders, ensuring a shared understanding of how the system is built and how its components interact. It also supports implementation, testing, and future improvements by clearly outlining design decisions and system behavior.

1.2 Scope of the system

The scope of the *BattleBot* system includes the design, development, and implementation of an autonomous robotic vehicle based on the Arduino platform. The system is intended to complete a relay-style course and participate in competition-based tasks using embedded hardware and software.

The *BattleBot* includes a microcontroller, motors, sensors, actuators, and supporting electronic components required for movement, obstacle detection, line following, and interaction with the environment. The software is responsible for processing sensor input, controlling the robot's behavior, and ensuring that the robot can operate autonomously during the assigned tasks.

This technical design covers the internal architecture of the robot, including hardware structure, software modules, control logic, and integration between physical and digital components. It also includes testing-related considerations needed to validate the correct operation of the system.

1.3 Clients

This document is intended for all stakeholders involved in the BattleBot project.

Primary stakeholders

- **NHL Stenden lecturers and module coordinators**
  Responsible for assessing the technical design and final product
- **Project supervisors**
  Provide guidance, feedback, and evaluation during development
- **Student development team**
  Use this document as a reference for implementation, testing, and collaboration

Secondary stakeholders

- **Competition organizers / evaluators**
  Interested in the robot's performance during the relay competition
- **Team members from other subgroups**
  Depend on correct behavior for relay interaction between robots

The document ensures that all stakeholders have a shared understanding of the system design, functionality, and technical decisions.

2. Project overview

2.1 System Description

The *BattleBot* system is an autonomous robotic vehicle designed to participate in a relay-based competition. The robot is built around an embedded system using an Arduino Nano microcontroller, which controls all hardware components and executes the programmed behavior.

The system integrates multiple sensors, actuators, and control logic to enable the robot to navigate a predefined course, detect obstacles, and perform specific tasks without human intervention. Key hardware components include DC motors for movement, a servo motor for the gripper mechanism, an ultrasonic sensor for distance measurement, and a line sensor array for path detection.

The software component of the system is developed in C/C++ using the Arduino IDE. It is responsible for reading sensor data, processing inputs, making decisions, and controlling the actuators accordingly. The system operates in a continuous loop, allowing real-time response to environmental changes.

The *BattleBot* is designed as a self-contained system, meaning all processing and decision-making occur on the onboard microcontroller. The robot must function

reliably under competition conditions, completing tasks such as line following, obstacle avoidance, and interaction with objects as part of a relay sequence.

Overall, the system combines hardware and software into a cohesive solution that demonstrates core principles of embedded systems, including real-time control, hardware interfacing, and autonomous operation.

2.2 Goals and objectives

The main goal of the *BattleBot* project is to design and develop a fully functional autonomous robot that can successfully complete a relay-based course and perform assigned tasks in a competitive environment.

To achieve this, the following objectives are defined:

- Develop a working embedded system using the Arduino platform
- Program the robot in C/C++ to operate autonomously without human intervention
- Enable accurate line following using sensor input
- Implement obstacle detection and avoidance using the ultrasonic sensor
- Control motors and actuators to ensure stable and efficient movement
- Integrate all hardware components into a reliable and safe system
- Test and optimize the robot's performance for speed and accuracy
- Ensure the system meets the functional requirements defined in the project brief

Additionally, the project aims to strengthen practical skills in embedded programming, system integration, and project-based teamwork. The final result should be a robust and efficient robot capable of completing its tasks consistently under competition conditions.

2.3 Development and testing platform

Development Environment

- **Arduino IDE 1.8.x**
  Used for writing, compiling, and uploading C/C++ code to the Arduino Nano
- **Programming Language: C/C++**
  Used for embedded system development
- **Git (version control)**
  Used for managing code versions, collaboration, and tracking changes

Hardware Platform

- Arduino Nano microcontroller
- Sensors (line sensor, ultrasonic sensor, encoders)
- Actuators (DC motors, servo motor, LEDs)
- Motor driver and PCB
- Power supply (power bank with separated logic and motor power)

Testing Environment

- **Small test maze**
  Used for initial testing and debugging of individual functions
- **Full-size competition maze**
  Used for integration testing and final validation
- **Relay setup with multiple robots**
  Used to test coordination and handover between robots

Measurement and Debugging Tools

- Serial Monitor (Arduino IDE) for debugging output
- Multimeter for voltage and continuity checks
- Visual observation of robot behavior
- Timing measurements for performance evaluation

3. Requirements

3.1 Functional requirements

The *BattleBot* system must meet the following functional requirements to operate correctly within the project scope and competition environment:

- The system must be able to operate fully autonomously without human interaction during execution
- The robot must be able to follow a line using the line sensor array
- The system must detect obstacles using the ultrasonic sensor
- The robot must avoid collisions with objects
- The system must control the speed and direction of the motors
- The robot must be able to start, stop, and navigate the course as required
- The system must process sensor input in real time and respond accordingly
- The robot must be able to complete its assigned relay task within the competition
- The system must control the servo motor to operate the gripper (if required

by the task)
- The robot must provide visual feedback using LEDs or indicators (optional but recommended)
- The system must be able to handle multiple tasks sequentially during operation

All functional requirements are focused on ensuring that the robot performs its intended tasks reliably and efficiently within the competition setting.

3.2 Non-functional requirements

The *BattleBot* system must meet the following non-functional requirements to ensure reliability, performance, and usability:

- The system must operate in real time with minimal delay in processing sensor input and controlling actuators
- The robot must be reliable and function consistently throughout the entire course without failure
- The system must be stable and avoid unexpected behavior such as crashes or uncontrolled movement
- The software must be modular and structured to support maintainability and future improvements
- The code must follow basic coding standards and be readable and well-documented
- The system must be efficient in terms of resource usage (CPU and memory limitations of the microcontroller)
- The robot must complete tasks within acceptable time limits to remain competitive
- The hardware setup must be safe and prevent damage to components or surroundings
- The system must be easy to test, debug, and modify during development
- The design must comply with given project constraints and hardware limitations

These requirements ensure that the system is not only functional but also robust, maintainable, and suitable for real-world operation in a competitive environment.

3.3 Constrains and assumptions

Constraints

The development of the *BattleBot* system is subject to the following constraints:

- The system must be built using the provided hardware, including the Arduino Nano microcontroller
- The software must be developed using C/C++ within the Arduino IDE
- Only approved components and suppliers may be used for additional hardware
- The robot must operate within the physical limits of the provided chassis and components
- Power supply is limited to the provided battery/power bank
- The system must comply with safety requirements and competition rules
- Development must be completed within the given project timeline and deadlines
- The robot must function without external computing resources (no cloud or external processing)

Assumptions

The following assumptions are made during the design and development of the system:

- All hardware components are functioning correctly unless stated otherwise
- The competition environment (track, obstacles, lighting conditions) remains within expected parameters
- Sensors provide sufficiently accurate data for navigation and decision-making
- The robot operates on a predefined course layout
- Team members have basic knowledge of programming and electronics
- Required software tools (Arduino IDE, documentation) are available and functional
- Testing conditions are representative of the actual competition environment

These constraints and assumptions define the boundaries within which the *BattleBot* system is designed and implemented.

3.4 requirement prioritization

Must Have (Critical requirements)

- The robot must operate fully autonomously without human intervention
- The robot must follow a line accurately using the line sensor array
- The robot must detect and avoid obstacles using the ultrasonic sensor
- The robot must control motor speed and direction reliably
- The robot must complete its assigned relay task within the competition

Should Have (Important but not critical)

- The robot should use encoder feedback for improved movement accuracy
- The robot should provide stable and consistent performance across multiple runs
- The robot should include a structured and modular software design
- The robot should handle state-based behavior (e.g., idle, active, obstacle detected)

Could Have (Optional enhancements)

- LED indicators for status feedback and debugging
- Additional optimization for speed and efficiency
- Improved gripper precision using fine servo control

Won't Have (Out of scope for this project)

- Wireless communication or remote control features
- External processing (e.g., cloud or external computers)
- Advanced AI or machine learning functionality

4. System Architecture

4.1 Overview

The *BattleBot* system is based on a layered embedded architecture that integrates hardware and software components into a single autonomous system. The architecture is centered around the Arduino Nano microcontroller, which acts as the main control unit responsible for processing inputs and controlling outputs.

At a high level, the system can be divided into three main layers:

- **Input Layer** – consists of sensors that collect data from the environment, such as the line sensor (for path detection), ultrasonic sensor (for obstacle

detection), and rotation sensors (for movement feedback)
- **Control Layer** – implemented in software on the microcontroller, responsible for processing sensor data, making decisions, and executing control logic
- **Output Layer** – includes actuators such as DC motors (movement), servo motor (gripper), and LEDs (visual feedback)

The system operates in a continuous loop where sensor data is read, processed, and translated into actions. This allows the robot to react in real time to changes in its environment.

The architecture follows a modular design approach, where each component (sensors, motor control, decision logic) is separated into logical units. This improves maintainability, simplifies debugging, and allows for easier future extensions.

Overall, the architecture ensures a clear separation between sensing, decision-making, and actuation, enabling the *BattleBot* to function as a reliable and autonomous embedded system.

4.2 High-level design

The design follows a **sense–process–act** model:

- **Sense** – The robot collects data from sensors, including the line sensor for path tracking, the ultrasonic sensor for obstacle detection, and rotation sensors for movement feedback
- **Process** – The microcontroller processes the sensor data using programmed logic and decision-making algorithms
- **Act** – Based on the processed data, the system sends control signals to actuators such as motors, the servo gripper, and LEDs

The software is divided into logical modules to separate responsibilities:

- **Sensor Module** – Handles reading and filtering data from all sensors
- **Control Module** – Contains the main decision-making logic (e.g., line following, obstacle avoidance)
- **Motor Control Module** – Manages speed and direction of the DC motors using PWM signals
- **Actuator Module** – Controls additional components such as the servo motor and LEDs

The system runs in a continuous loop (loop() function in Arduino), ensuring real-time responsiveness. Time-based functions (such as millis()) may be used instead of blocking delays to improve performance and allow multiple processes to run simultaneously.

5. Hardware Design

5.1 Robot components

Arduino Nano (Microcontroller)
The core of the system, responsible for executing the program logic, processing sensor data, and controlling all connected components.

Chassis (Robot Frame)
A mechanical structure that holds all components in place, including motors, sensors, PCB, and power supply. It provides stability and mobility.

DC Motors (2x)
Used to drive the wheels of the robot. They enable forward, backward, and turning movements.

Motor Driver Module
Controls the DC motors by regulating speed and direction based on signals from the microcontroller.

Wheels
Attached to the motors to provide traction and movement across the surface.

Servo Motor (Gripper Mechanism)
Enables precise control of the gripper for interacting with objects during tasks.

Line Sensor Array
Detects the line on the track and allows the robot to follow a predefined path.

Ultrasonic Distance Sensor
Measures the distance to obstacles, helping the robot avoid collisions.

Rotation Sensors (Encoders)
Provide feedback on wheel rotation, which can be used for speed control and distance measurement.

LED Indicators
Used for visual feedback, such as status indication or debugging signals.

Push Buttons (Optional)
Allow manual interaction, such as starting or resetting the robot during testing.

Power Supply (Battery / Powerbank)
Provides electrical energy to the entire system. Includes separate control for logic and motors.

PCB (Printed Circuit Board)
Serves as the central connection hub for all electronic components, ensuring proper wiring and power distribution.

5.2 Power System

The system is powered using a rechargeable power bank, which provides a 5V supply to the Arduino Nano and connected electronics. The power distribution is managed through the PCB, ensuring that each component receives the required voltage and current.

The design separates power for logic and motors to improve stability and safety:

- **Logic Power (MAIN_PWR)**
  Supplies power to the Arduino Nano, sensors, and low-power components. This ensures stable operation of the control system.
- **Motor Power (MOTOR_PWR)**
  Supplies power to the DC motors through the motor driver. Motors typically require higher current and may introduce electrical noise, so separating this from the logic circuit prevents interference.

Two physical switches are used to control power:

- One switch enables or disables the main electronics (logic power)
- One switch controls the motor power independently

This setup allows safe testing and debugging by enabling the system without activating the motors.

The system must ensure that the power supply is sufficient for peak loads, especially when motors and multiple components are active simultaneously. Proper grounding and wiring through the PCB help maintain stable operation

and reduce the risk of voltage drops or short circuits.

5.3 Hardware Constrains

Limited Processing Power
The Arduino Nano has limited CPU performance and memory (Flash and SRAM), which restricts the complexity of algorithms and the number of simultaneous tasks.

Limited I/O Pins
The number of available input/output pins on the microcontroller is restricted, requiring efficient allocation and possible sharing of pins between components.

Power Limitations
The power bank provides limited current, especially under high load when motors and multiple components are active at the same time.

Motor Interference
DC motors can introduce electrical noise, which may affect sensor readings and system stability if not properly managed.

Physical Size Constraints
All components must fit within the dimensions of the robot chassis, limiting expansion and placement options.

Weight Constraints
Excessive weight can negatively impact speed, stability, and motor performance.

Sensor Limitations
Sensors such as the ultrasonic sensor and line sensor have limited accuracy and range, which may affect detection reliability.

Wiring and PCB Layout
Incorrect wiring or poor connections can lead to system failures, making careful hardware integration essential.

Component Compatibility
All hardware components must be compatible with the Arduino Nano in terms of voltage levels and communication interfaces.

Environmental Conditions
Changes in lighting, surface quality, or obstacles may influence sensor

performance and overall system behavior.

5.4 I/O Lists

Robot 1

Physical Inputs

Name	Type	Pin	Description
Ultrasonic Trigger	Digital Output	7	Sends trigger pulse for distance measurement
Ultrasonic Echo	Digital Input	8	Receives reflected ultrasonic signal
Line Sensor 0	Analog Input	A0	Left-most line detection
Line Sensor 1	Analog Input	A1	Line detection
Line Sensor 2	Analog Input	A2	Line detection
Line Sensor 3	Analog Input	A3	Line detection
Line Sensor 4	Analog Input	A4	Line detection
Line Sensor 5	Analog Input	A5	Line detection
Line Sensor 6	Analog Input	A6	Line detection
Line Sensor 7	Analog Input	A7	Right-most line

detection

detection

Physical Outputs

Name	Type	Pin	Description
Motor Left Forward	PWM Output	5	Drives left wheel forward
Motor Left Backward	PWM Output	3	Drives left wheel backward
Motor Right Forward	PWM Output	6	Drives right wheel forward
Motor Right Backward	PWM Output	11	Drives right wheel backward
Servo Gripper	PWM Output	9	Opens and closes gripper
NeoPixel LED	Digital Output	12	Status indication / lighting

Robot 2

Physical Inputs

Name	Type	Pin	Description
Front Ultrasonic Trigger	Digital Output	12	Sends trigger pulse for front distance measurement
Front Ultrasonic Echo	Digital Input	13	Receives reflected front ultrasonic signal
Left Ultrasonic Trigger	Digital Output	11	Sends trigger pulse for left-side distance measurement
Left Ultrasonic Echo	Digital Input	8	Receives reflected left ultrasonic signal
Line Sensor 0	Analog Input	A0	Left-most line detection
Line Sensor 1	Analog Input	A1	Line detection

Line Sensor 2	Analog Input	A2	Line detection
Line Sensor 3	Analog Input	A3	Line detection
Line Sensor 4	Analog Input	A4	Line detection
Line Sensor 5	Analog Input	A5	Line detection
Line Sensor 6	Analog Input	A6	Line detection
Line Sensor 7	Analog Input	A7	Right-most line detection

Physical Outputs

Name	Type	Pin	Description
Motor Left Forward	PWM Output	3	Drives left wheel forward
Motor Left Reverse	PWM Output	5	Drives left wheel backward
Motor Right Forward	PWM Output	6	Drives right wheel forward
Motor Right Reverse	PWM Output	9	Drives right wheel backward
Servo Gripper	PWM Output	10	Opens and closes gripper
Buzzer	Digital Output	7	Provides audio feedback / warning signal

6. Software Design

6.1 Software architecture

The architecture is divided into the following main layers:

- **Hardware Abstraction Layer (HAL)**
  Provides an interface between the software and hardware components. It handles low-level operations such as reading sensors and controlling actuators, allowing higher-level logic to remain hardware-independent.
- **Application Logic Layer**
  Contains the main decision-making logic of the system, including line following, obstacle avoidance, and task execution. This layer processes input data and determines the robot's behavior.
- **Control Layer**
  Responsible for translating decisions into actions, such as setting motor speed, direction, and actuator states.

The software is further organized into modules:

- **Sensor Module** – Reads and processes data from all sensors
- **Motor Control Module** – Controls speed and direction using PWM
- **Actuator Module** – Manages servo motor and LEDs
- **Main Controller** – Coordinates all modules and maintains system flow

To improve performance, the system avoids blocking functions where possible and may use time-based logic (e.g., millis()) to handle multiple operations simultaneously.

6.2 Modules and Components

Main Software Modules

- Main Module
  Controls the overall execution of the program. It initializes all components in the setup() function and continuously coordinates system behavior in the loop() function.

- Sensor Module
  Handles input from the line sensor, ultrasonic sensor, and rotation sensors. It is responsible for collecting and preparing data for the control logic.
- Motor Control Module
  Manages the DC motors by controlling speed and direction through PWM signals and motor driver outputs.
- Servo Control Module
  Controls the gripper servo motor and sets its position when object interaction is required.
- Decision Logic Module
  Processes sensor data and determines the robot's actions, such as following a line, avoiding obstacles, or executing a relay task.
- LED / Status Module
  Controls visual feedback components such as standard LEDs and NeoPixels to indicate robot states, events, or debugging information.

Libraries and Software Components

- Arduino Core Library
  Provides the standard Arduino functions such as setup(), loop(), digitalRead(), digitalWrite(), analogRead(), and analogWrite().
- Adafruit NeoPixel Library
  Used to control the NeoPixel LEDs on the BattleBot. This library enables individual control of LED color and brightness, making it possible to display status signals, warnings, or animation effects.

6.3 Control logic

The control process follows a repeated cycle of **read – evaluate – act**:

- First, the system reads data from all relevant sensors
- Next, the software evaluates the current situation based on predefined conditions
- Finally, the system performs the required action by controlling motors, the servo motor, or status indicators

The main behaviors included in the control logic are:

- **Line Following**
  The robot uses the line sensor array to detect its position relative to the track line. Based on the sensor readings, the control logic adjusts motor

speed and direction to keep the robot on course.

- **Obstacle Detection and Avoidance**
  The ultrasonic sensor continuously measures the distance to objects in front of the robot. If an obstacle is detected within a defined threshold, the robot slows down, stops, or changes behavior to avoid collision.
- **Movement Control**
  The control logic determines whether the robot should move forward, turn left, turn right, stop, or reverse. These actions are translated into specific motor commands.
- **Task Execution**
  If the relay task requires interaction with objects, the control logic activates the servo motor to operate the gripper at the correct moment.
- **State Handling**
  The robot may operate using defined states such as *idle*, *line following*, *obstacle detected*, *task active*, and *stop*. This makes the behavior easier to manage and debug.

To improve responsiveness, the control logic should avoid long blocking delays. Instead, timed behavior can be managed using non-blocking functions such as millis(). This allows the robot to monitor sensors and respond to changes in real time.

7. Processes and Workflow

7.1 Battle Scenario Workflow

The *BattleBot* system is designed as a relay-based workflow involving three autonomous robots that cooperate by transferring a cone from one robot to the next. Each robot is responsible for completing a specific section of the course before handing over the cone to the following robot. The workflow begins when the start flag is raised, which serves as the trigger for the first robot to begin operation.

At the start of the scenario, **Robot 1** detects the start condition and begins its task. It approaches the cone, uses its gripper mechanism to pick it up, and then navigates through its assigned section of the map. After reaching the designated handover area, Robot 1 places the cone in the correct position and moves away from the drop-off zone to clear space for the next robot.

Once the cone has been placed, **Robot 2** detects its presence using its sensors. This detection acts as the trigger for Robot 2 to begin operation. Robot 2 approaches the cone, picks it up with its gripper, and travels through its own assigned section of the course. After completing its route, it places the cone in the next designated transfer area and then moves away from the cone.

After Robot 2 has completed its task, **Robot 3** detects the cone in the same way. It then moves toward the cone, picks it up, and navigates through the final section of the map. At the end of its route, Robot 3 places the cone in the final target location. Unlike the previous robots, Robot 3 does not move away afterward, but instead stops completely, marking the end of the relay process.

This workflow ensures sequential cooperation between the three robots. Each robot starts only after the previous one has successfully completed its task and transferred the cone. In this way, the full battle scenario is completed as a coordinated autonomous relay system.

8. Testing and validation

8.1 Test strategy

The testing strategy for the *BattleBot* project is based on gradual validation of the system in increasingly realistic conditions. Testing begins in a small test maze, where individual functions and basic robot behavior can be checked in a controlled environment. After this, the robots are tested in the full-size maze to evaluate performance under conditions that are closer to the final competition setup.

The purpose of this approach is to first identify and fix basic issues on a smaller scale before moving to more complex and realistic testing. This reduces risk and makes debugging more efficient.

Testing is performed in two ways:

- **Team testing**
  The full team tests the complete relay scenario together. This is used to verify coordination between the three robots, cone handover, timing, and overall system integration.
- **Individual part testing**
  Each part of the system is also tested separately. Team members test their

own robot section, functions, and assigned tasks individually to make sure that each robot works correctly on its own before integration.

The testing process focuses on the following areas:

- correct sensor behavior
- reliable line following
- obstacle detection
- cone pickup and placement
- correct start and stop behavior
- relay handover between robots
- stability and repeatability of performance

9. Standards and Guidelines

9.1 Coding standards

The software for the *BattleBot* project should follow clear and consistent coding standards to improve readability, maintainability, and teamwork. Since the system is developed in C/C++ using the Arduino environment, the code should be structured in a simple and organized way so that all team members can understand, test, and modify it when needed.

The following coding standards are used in this project:

- Code should be written in a clear and consistent style
- Variable names and function names should be descriptive and meaningful
- Naming conventions should remain consistent throughout the project
- Each function should have one clear responsibility
- Repeated code should be avoided where possible
- Comments should be added where necessary to explain important logic or hardware-specific behavior
- Unused code, variables, and libraries should be removed
- Indentation and formatting should be consistent in all files
- Constants should be used instead of hardcoded values where possible
- Code should be divided into logical modules such as sensors, motor control, and decision logic
- Blocking delays should be avoided when real-time responsiveness is required
- All team members should use the same version control practices when

updating code

9.2 Safety Requirements

The *BattleBot* system must meet basic safety requirements to ensure safe operation during development, testing, and competition. These requirements are intended to prevent damage to hardware, the environment, and users.

- The robot must not collide with objects, obstacles, or other robots during operation
- The system must include reliable obstacle detection and stopping behavior
- Motors must be controlled to prevent uncontrolled or sudden movements
- A physical power switch must be available to quickly disable the robot if needed
- The motor power and logic power must be separated to reduce risk of malfunction
- All wiring must be properly connected and insulated to prevent short circuits
- Components must be securely mounted to avoid loose parts during movement
- The robot must not overheat during operation
- Electrical components must be used within their specified voltage and current limits
- The system must behave predictably and avoid unstable or unsafe actions
- Testing should be performed in a controlled environment to reduce risk of accidents

10. References to related documents

This technical design builds upon and is supported by several related documents and resources:

- **Requirements Analysis**
  Defines the functional and non-functional requirements of the system

- **BattleBot Portfolio**
  Contains weekly assignments, code, test results, and reflections
- **Test Matrix**
  Defines test cases, expected results, and validation criteria
- **Module Book – Project BattleBot (2025–2026)**
  Provides project requirements, learning outcomes, and assessment criteria
- **The Ultimate BattleBot Guide**
  Technical manual for hardware components and system setup
- **Arduino Documentation**
  Reference for programming functions, libraries, and hardware interfacing